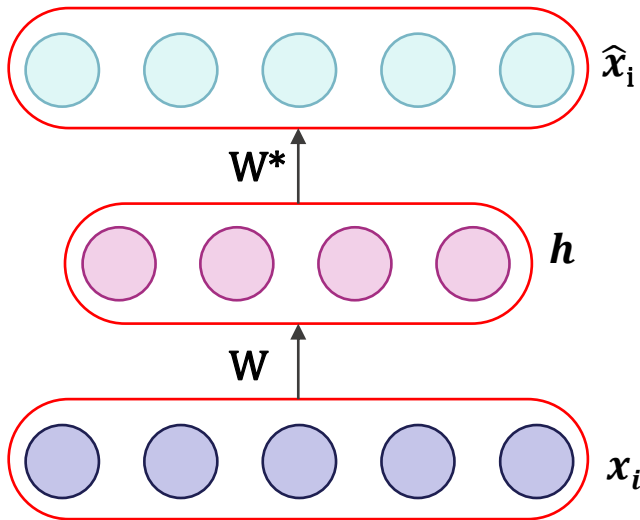


QpiAI™ - Joint AI Training and Certification Program

Autoencoders

Sachin Kumar

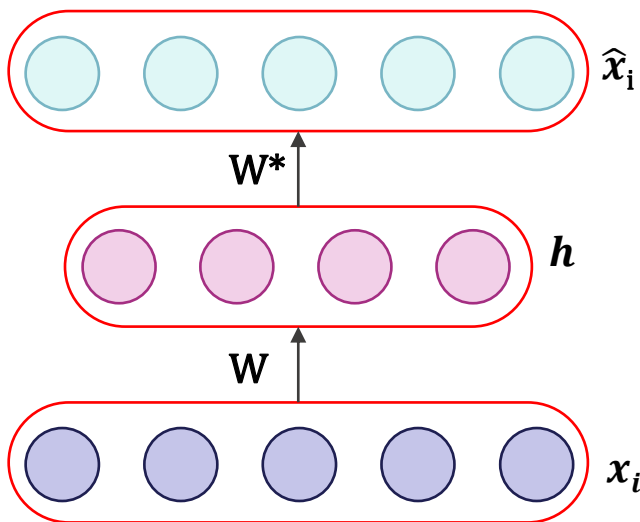
Introduction to Autoencoders



- An autoencoder is a special type of feed forward neural network which does the following
- Encodes its input x_i into a hidden representation h
- Decodes the input again from this hidden representation
- The model is trained to minimize a certain loss function which will ensure that $\hat{x}_i$ is close to x_i

$$\begin{aligned} h &= g(Wx_i + b) \\ \hat{x}_i &= f(W^*h + c) \end{aligned}$$

Introduction to Autoencoders: Undercomplete



$$\begin{aligned} h &= g(Wx_i + b) \\ \hat{x}_i &= f(W^*h + c) \end{aligned}$$

- Let us consider the case where $\dim(\mathbf{h}) < \dim(\mathbf{x}_i)$
- If we are still able to reconstruct $\hat{x}_i$ perfectly from $\mathbf{h}$, then what does it say about $\mathbf{h}$?
- $\mathbf{h}$ is a loss-free encoding of $\mathbf{x}_i$. It captures all the important characteristics of $\mathbf{x}_i$
- Analogous with PCA

An autoencoder where $\dim(\mathbf{h}) < \dim(\mathbf{x}_i)$ is called an under complete autoencoder

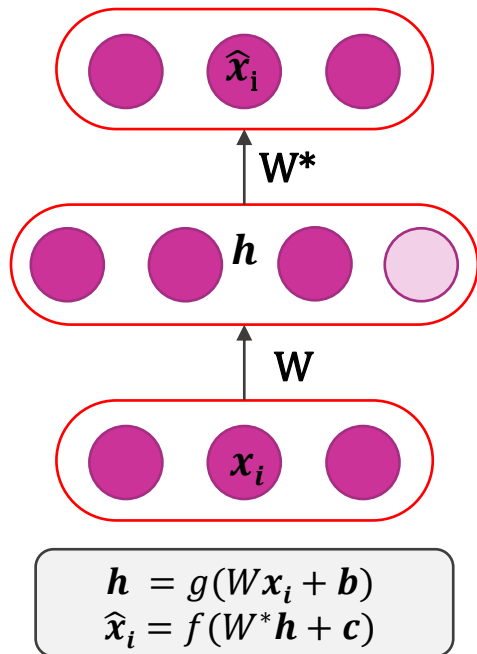
Remember

The encoder of a linear autoencoder is equivalent to PCA if we

- Use a linear encoder
- Use a linear decoder
- Use a squared error loss function
- And normalize the inputs to

$$\hat{x}_{ij} = \frac{1}{\sqrt{m}} \left(x_{ij} - \frac{1}{m} \sum_{k=1}^m x_{kj} \right)$$

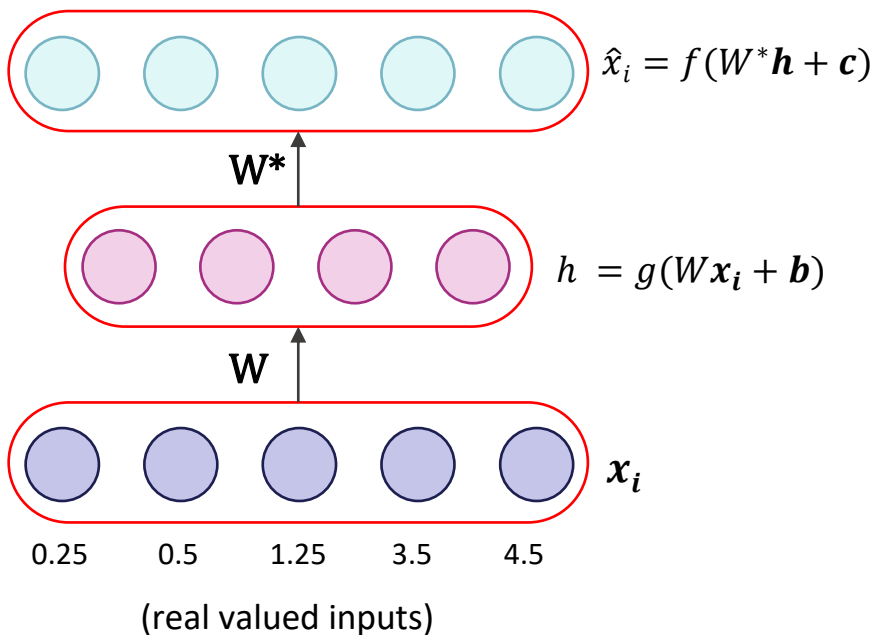
Introduction to Autoencoders: Overcomplete



An autoencoder where $\dim(\mathbf{h}) < \dim(\mathbf{x}_i)$ is called an over complete autoencoder

- Let us consider the case where $\dim(\mathbf{h}) \geq \dim(\mathbf{x}_i)$
- In such a case the autoencoder could learn a trivial encoding by simply copying $\mathbf{x}_i$ into $\mathbf{h}$ and then copying $\mathbf{h}$ into $\hat{\mathbf{x}}_i$
- Such an identity encoding is useless in practice as it does not really tell us anything about the important characteristics of the data

Choice of Activation Functions



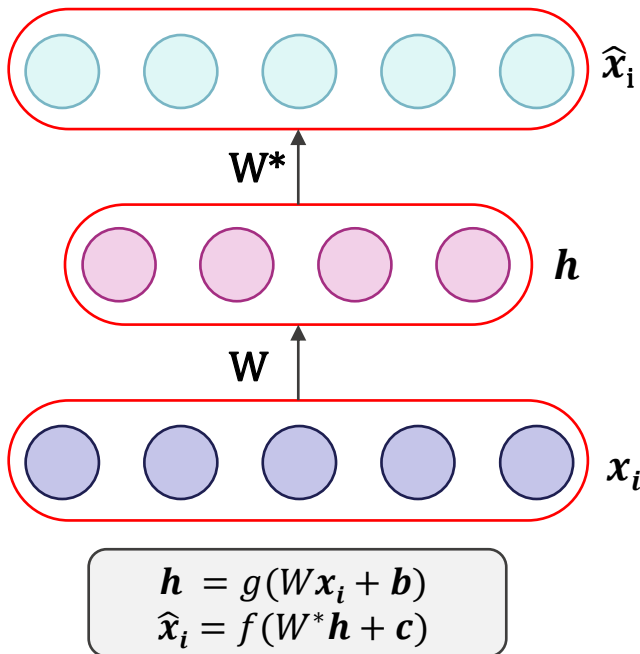
- Suppose all our inputs are real (each $x_{ij} \in \mathbb{R}$)
- Which of the following functions would be most apt for the decoder?

$$\begin{aligned}\hat{x}_i &= \tanh(W^*h + c) \\ \hat{x}_i &= W^*h + c \\ \hat{x}_i &= \text{logistic}(W^*h + c)\end{aligned}$$

They will restrict the reconstructed $\hat{x}_i$ to lie between $[0,1]$ or $[-1,1]$ whereas we want $\hat{x}_i \in \mathbb{R}^n$

Function g is typically chosen as the sigmoid function

Real Valued Inputs



- Consider the case when the inputs are real valued
- The objective of the autoencoder is to reconstruct $\hat{x}_i$ to be as close to x_i as possible
- This can be formalized using the following objective function:

$$\min_{W, W^*} \frac{1}{m} \sum_{i=1}^m \sum_{j=1}^n (\hat{x}_{ij} - x_{ij})^2$$

i. e., $\min_{W, W^*} \frac{1}{m} \sum_{i=1}^m (\hat{x}_i - x_i)^T (\hat{x}_i - x_i)$

- We can then train the autoencoder just like a regular feedforward network using backpropagation

Autoencoder Loss Functions

For binary inputs:

$$l(f(X)) = - \sum_k (x_k \log(\hat{x}_k) + (1 - x_k) \log(1 - \hat{x}_k))$$

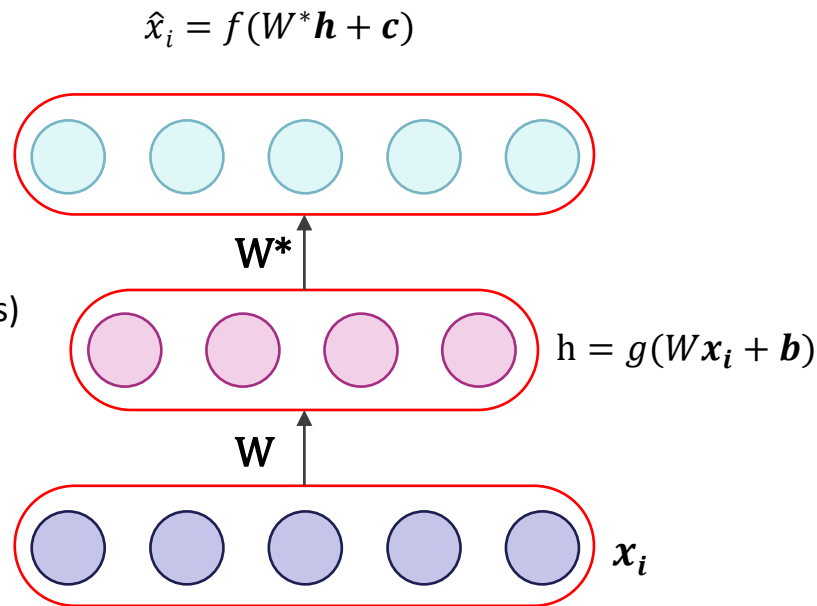
Cross-Entropy (more precisely: Sum of Bernoulli Cross-Entropies)

- For real-valued inputs:

$$l(f(X)) = \frac{1}{2} \sum_k (\hat{x}_k - x_k)^2$$

Sum of squared differences (squared Euclidean distance)

We use a linear activation function at the output

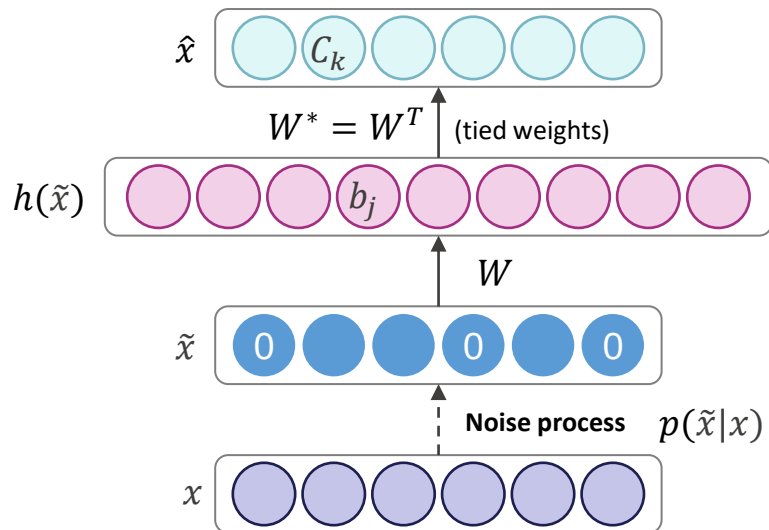


Applications: Denoising Autoencoder

Idea: representation should be robust to introduction of noise:

- Random assignment of subset of inputs to 0, with probability v
- Gaussian additive noise

Reconstruction $\hat{x}$ computed from the corrupted input $\tilde{x}$



Denoising Autoencoder

Idea: representation should be robust to the introduction of noise

- Random assignment of subset of inputs to 0, with probability v
- Gaussian additive noise

Reconstruction $\hat{x}$ computed from the corrupted input $\tilde{x}$

Loss function compares $\hat{x}$ reconstruction with the **noiseless input x**

